

[PROJECT TITLE]

Customer Journey Analytics Platform

Project Submission Report

Submitted by

[Team Member Name 1]

[Team Member Name 2]

[Team Member Name 3]

[Team Member Name 4]

[Course Name / Code]

[Institution Name]

[Submission Date]

Table of Contents

1. Executive Summary

This report documents the design, implementation, and commercial positioning of a Customer Journey Analytics (CJA) platform — a behavioural intelligence system built for e-commerce website owners. Unlike conventional web analytics tools that report what already happened, this platform combines session-level behavioural tracking, visual heatmap analysis, machine-learning-based drop-off prediction, and natural-language sentiment analysis to surface which visitors are at risk of abandoning a purchase, and why.

The system was developed as an academic project demonstrating an end-to-end data pipeline: from a lightweight client-side tracking script, through an asynchronous ingestion API, a session-aggregation layer, an analytics engine, and finally a five-panel administrative dashboard. The platform was further evolved during development from a single-tenant proof of concept into a multi-tenant, loosely-coupled architecture suitable for onboarding real, independent client websites — a deliberate design decision intended to demonstrate not just technical correctness but commercial viability.

Key outcomes of this project

- A complete 8-stage data pipeline, from client-side event capture through to ML-driven insight generation
- A standalone, framework-agnostic tracking script (tracker.js) that integrates into any website via a single script tag — validated against two structurally distinct demo sites
- Multi-tenant data isolation using a `site_id` key, enabling one shared backend to serve multiple independent client websites simultaneously
- A five-panel admin dashboard (Funnel, Heatmap, Session Analytics, Risk Prediction, Sentiment Insights) with live site-switching
- A documented integration process and business model, positioning the platform competitively against existing tools such as Google Analytics, Hotjar, and Mixpanel

This report is organised to walk a reviewer through the system in the same order it was built: architecture and data flow first, followed by the engineering decisions behind multi-tenancy and integration, then the commercial case for the platform, and finally an honest account of its current limitations and a roadmap for future work.

2. System Architecture

The platform is composed of two independently deployable applications sharing a common backend and database: a client-facing tracking layer embedded in the e-commerce website, and an administrative dashboard used by the site owner to view analytics. This separation mirrors the architecture used by production analytics platforms such as Hotjar and Mixpanel, where the tracked website and the analytics console are entirely distinct applications connected only through an API.

2.1 Technology Stack

Layer	Technology	Purpose
Frontend (storefront)	React / HTML / JavaScript	The e-commerce website being tracked
Tracking layer	Vanilla JavaScript (tracker.js)	Captures behavioural events client-side
Backend API	Node.js + Express	Ingests events, serves analytics endpoints
Database	PostgreSQL	Stores raw events, sessions, and derived features
Admin dashboard	React	Visualises analytics across five panels
ML — drop-off prediction	XGBoost	Predicts session-level abandonment risk
ML — sentiment analysis	NLP (text classification)	Scores customer feedback sentiment

2.2 The Eight-Stage Data Pipeline

Data moves through the system in eight distinct stages, from a raw click on the storefront to an actionable insight on the admin dashboard.

Stage 1 — Session initialisation

When a visitor loads the tracked website, the tracking script generates a UUID `session_id`, persisted in the browser for the duration of that browsing session. This ID anchors every subsequent event to a single, coherent visitor journey.

Stage 2 — Client-side event capture

The tracking script listens for clicks (with pixel coordinates), scroll depth percentage, page views, and debounced mouse movement. Each event is serialised into a compact JSON payload containing the session ID, event type, coordinates where applicable, the current page, and a timestamp.

Stage 3 — Asynchronous ingestion

Events are batched and sent via a non-blocking HTTP POST to the `/api/track` endpoint. The Node.js backend validates each payload and writes it to the `raw_events` table in PostgreSQL. Because this call is asynchronous and uses the browser's `sendBeacon` API on page exit, the visitor never experiences latency on the storefront, even under heavy tracking load.

Stage 4 — Sessionisation

A scheduled aggregation job groups raw events by `session_id` and computes session-level features: total duration, number of pages visited, the deepest funnel stage reached, average scroll depth, and click count. These features are written to a dedicated `session_features` table, decoupling expensive aggregation from real-time dashboard queries.

Stage 5 — Funnel and heatmap computation

The analytics engine queries session data to compute conversion rates between each funnel stage, and aggregates click and cursor coordinates into heatmap density matrices per page. Scroll depth distributions are similarly aggregated to show where visitor attention drops off on a page.

Stage 6 — Parallel machine learning inference

Two independent ML processes run against the processed data. An XGBoost classifier consumes the `session_features` vector to output a drop-off probability, mapped to a High / Medium / Low risk label. Simultaneously, an NLP pipeline processes any free-text customer feedback to produce a sentiment score (positive, neutral, or negative).

Stage 7 — Persisting derived insights

Both the risk score and the sentiment label are written back to PostgreSQL alongside their source session. This is a deliberate architectural choice: pre-computing ML outputs means the dashboard never needs to re-run inference on every page load — it simply queries pre-existing, indexed results.

Stage 8 — Dashboard consumption

The admin dashboard, a separate authenticated application, reads from a family of GET `/api/analytics/*` endpoints. Each of its five panels — Funnel, Heatmap, Session Analytics, Risk Prediction, and Sentiment Insights — independently fetches and renders its own slice of the processed data.

Architectural assessment: the pipeline's separation of concerns — particularly the decision to pre-compute ML outputs rather than run inference on-demand, and to decouple the tracked website from the admin dashboard — reflects patterns used in production-grade analytics systems rather than typical academic prototypes. This separation is what subsequently made multi-tenancy possible without a structural rewrite (see Section 3).

3. System Design Decisions: From Single-Tenant Prototype to Loosely-Coupled Platform

The platform was initially built and validated against a single dummy e-commerce website developed in-house. While sufficient to prove the analytics and ML pipeline worked end-to-end, this design was tightly coupled: the tracking script was embedded directly inside the demo site's codebase, funnel stage names were hardcoded to that site's specific page structure, and there was no mechanism to distinguish one client's data from another's. A core engineering objective of this project was to evolve this prototype into an architecture that any independent e-commerce website could integrate with, without modification to our codebase and without our codebase needing to know anything about theirs.

3.1 Problem Identification

Issue	Why it matters commercially
Hardcoded funnel page names	A real store's URL structure (e.g. /shop/item vs /products) would break funnel detection entirely.
Tracking script embedded in our codebase	An external site owner cannot install analytics that lives inside someone else's repository.
No tenant identity in the database	Without isolation, one client's heatmaps and session data would be visible to — or contaminated by — another's.
Single shared site assumption throughout the stack	The admin dashboard had no concept of "switching" between clients; it assumed exactly one website existed.

3.2 Solution: A Universal, Configurable Tracking Script

The tracking logic was extracted from the demo site into a standalone file, `tracker.js`, hosted independently of any client's website — conceptually identical to how Google Analytics or Hotjar distribute their tracking tags. A client integrates the platform by pasting a short snippet into the `<head>` of their site:

```
<script>
  window.CJA_CONFIG = {
    site_id: "client_unique_id",
    track_endpoint: "https://api.platform.com/api/track",
    funnel: ["/products", "/cart", "/checkout", "/order-complete"]
  };
</script>
<script src="https://cdn.platform.com/tracker.js" async></script>
```

This single change removes the dependency on our codebase entirely. The script is vanilla JavaScript with zero external dependencies, auto-initialises on page load, and works identically whether the host site is built with plain HTML, React, Next.js, or a platform such as Shopify or WooCommerce.

Making the funnel configurable, not hardcoded

Rather than assuming every store follows the same Product → Cart → Checkout → Payment structure, the funnel stages are now passed in as a JSON array at initialisation. The backend reads this configuration per `site_id`, meaning the same database schema and the same analytics logic can support an arbitrary number of differently structured client websites without any code changes.

3.3 Solution: Multi-Tenant Data Isolation

Rather than provisioning a separate database per client — an approach that becomes operationally unmanageable beyond a handful of clients, requiring duplicated migrations, backups, and scaling effort for each — a shared-schema, tenant-column pattern was adopted. This is the same pattern used by platforms such as Stripe, Hotjar, and Mixpanel internally.

A `site_id` column was added as a foreign key across every relevant table: `raw_events`, `sessions`, `session_features`, and `feedback`. Every query in the API layer filters by this column, so a single PostgreSQL instance can safely serve multiple independent clients, each seeing only their own data.

```
raw_events
+-----+-----+-----+-----+
| site_id | session_id | event_type | x | y |
+-----+-----+-----+-----+
| ecommerce_001 | uuid-aaa | click | 340 | 210 |
| demo_bookstore_002 | uuid-bbb | click | 120 | 80 |
+-----+-----+-----+-----+
```

This design was explicitly validated, not just assumed correct: a second demo site — a static HTML bookstore with a deliberately different URL structure and a different funnel configuration — was built and integrated using only the public `tracker.js` snippet. Verification queries confirmed that events from each site remained fully isolated under their respective `site_id`, with no cross-contamination.

3.4 Solution: A Site-Aware Admin Dashboard

The admin dashboard was updated to support a site selector, allowing an administrator managing multiple client websites to switch between them and have every one of the five analytics panels reload with that site's data. This required three considerations beyond a simple dropdown: avoiding stale data being shown while a new site's data loads, guarding against race conditions where a slow response for a previously selected site could overwrite a newly selected site's correct data, and providing clear empty/error states scoped to individual panels so that one site's missing data does not break the entire dashboard.

3.5 Requirements for Real-World Integration

Beyond what has already been implemented, the following requirements were identified as necessary for onboarding genuine, independent client websites in a production setting:

- A public-facing CDN hosting `tracker.js` with high availability and low latency, independent of any single client's infrastructure
- A client onboarding flow that issues a unique `site_id` and a default funnel configuration, without requiring engineering involvement per client
- Rate limiting and basic payload validation on the ingestion endpoint, to protect the shared database from a misbehaving or malicious integration
- HTTPS enforcement and CORS configuration scoped to registered client domains, to prevent the tracking endpoint from accepting arbitrary cross-origin traffic
- A lightweight verification/debug mode (already implemented in `tracker.js`) so a client can confirm correct installation without our team's involvement
- Native plugins for common platforms — a Shopify app and a WooCommerce plugin — that auto-inject the script and funnel configuration, removing manual integration steps entirely for the largest segment of e-commerce site owners

The shift from a single hardcoded demo to a configurable, multi-tenant architecture is the single most significant engineering change made during this project. It does not require a rewrite of the existing pipeline — every stage described in Section 2 continues to function unchanged — but it transforms the system from a working demonstration into something that resembles an onboardable product.

4. Business Model and Market Positioning

Although developed as an academic project, the platform was deliberately designed with a commercial audience in mind: e-commerce website owners who want to understand not just how many visitors their site receives, but which visitors are likely to abandon their purchase, and why. This section outlines how the platform is positioned relative to existing tools, and the routes available to monetise it.

4.1 Core Value Proposition

The platform's differentiation is that it combines four capabilities that are normally split across separate, specialised tools: session replay-style heatmaps, funnel conversion analysis, machine-learning-based drop-off prediction, and natural-language sentiment scoring of customer feedback. The headline pitch to a prospective client is straightforward: the platform does not simply report what happened on the website last week — it flags which current visitors are at risk of leaving before they complete a purchase, and surfaces the qualitative reasons (via sentiment analysis) that may explain why.

4.2 Competitive Landscape

Platform	Heatmaps	Funnel Analysis	ML Drop-off Prediction	Sentiment (NLP)
Google Analytics	No	Yes	No	No
Hotjar	Yes	Limited	No	No
Mixpanel	No	Yes	No	No
This Platform	Yes	Yes	Yes	Yes

Comparison reflects publicly known core feature sets of each platform at a general level; specific plans and add-ons may extend individual products' capabilities beyond what is shown here.

No single mainstream competitor combines all four capabilities natively. This is the platform's primary defensible position: a customer who currently needs Hotjar for heatmaps and a separate ML or BI tool for predictive insight could consolidate both into a single subscription.

4.3 Monetisation Models

Model A — Tiered SaaS subscription

The most direct analogue to Hotjar's own pricing model: a recurring monthly fee scaled to monthly tracked sessions and feature access.

Tier	Monthly sessions	Indicative price (INR/month)	Included features
Starter	Up to 10,000	₹999	Funnel + heatmap only
Pro	Up to 100,000	₹2,999	Adds drop-off risk prediction + sentiment
Enterprise	Custom	Custom	All features, dedicated support, custom funnels

Model B — Onboarding fee plus recurring licence

A one-time fee covering integration support (script installation, funnel configuration) combined with a smaller ongoing monthly fee for dashboard access. This model suits mid-market store owners who

value hands-on setup assistance over a fully self-service flow, and is common among Indian SaaS vendors targeting SMB e-commerce.

Model C — White-label for digital marketing agencies

Agencies managing multiple Shopify or WooCommerce clients could license a multi-site version of the dashboard under their own branding, reselling the underlying analytics and predictive insight as part of their own service offering to their end clients.

4.4 Go-to-Market Considerations

- Indian SMB e-commerce (Shopify/WooCommerce sellers) is a reasonable initial target segment given lower switching costs and high price sensitivity to the tiered model
- A self-service onboarding flow (sign up, receive site_id, paste script tag) is essential to scale beyond a handful of manually onboarded pilot clients
- Native platform plugins (see Section 3.5) would meaningfully reduce onboarding friction and should be prioritised ahead of a broader marketing push
- Case studies built from real pilot clients' before/after conversion data would be the strongest credibility asset for future sales conversations

5. Integration Approach: tracker.js in Detail

This section documents the standalone tracking script in full, as the integration mechanism is the platform's primary point of contact with a client's website and was treated as a first-class deliverable in its own right rather than an implementation detail.

5.1 Design Goals

- Zero dependencies — pure JavaScript, so it cannot conflict with any client's existing libraries or framework version
- Framework-agnostic — identical behaviour on static HTML, React, Next.js, Shopify, and WooCommerce
- Fail-silent — any internal error in the tracker must never break the host website, particularly its checkout flow
- Non-blocking — all network activity is asynchronous and never delays page rendering
- Minimal footprint — approximately 8KB, negligible impact on page load performance

5.2 Initialisation and Configuration

A client integrates the platform by setting a global configuration object before loading the script, then loading the script itself asynchronously:

```
<script>
  window.CJA_CONFIG = {
    site_id: "client_abc_123",
    track_endpoint: "https://api.platform.com/api/track",
    funnel: ["/products", "/cart", "/checkout", "/order-complete"],
    debug: false
  };
</script>
<script src="https://cdn.platform.com/tracker.js" async></script>
```

Parameter	Required	Description
site_id	Yes	Unique client identifier; tags every event for tenant isolation
track_endpoint	Yes	The ingestion URL events are POSTed to
funnel	Yes	Ordered array of URL paths defining the conversion journey; supports partial matching for dynamic routes (e.g. /product/:id)
debug	No	When true, logs every tracked event to the browser console for verification during setup

5.3 Events Captured

Event type	Data captured	Trigger
page_view	URL, timestamp	Page load and SPA route changes (popstate / pushState)
click	x, y coordinates, target element, page	Every click
scroll	Scroll depth percentage	Throttled to once per second
mouse_move	Cursor x, y	Debounced (~200ms), sampled rather than continuous

funnel_stage	Stage index, matched path	When the current URL matches a configured funnel path
--------------	---------------------------	---

5.4 Reliability Mechanisms

Batching

Events are accumulated client-side and sent as a batch every three seconds, or immediately once ten events have accumulated, whichever occurs first. This reduces request volume compared to sending one HTTP call per interaction.

Guaranteed delivery on page exit

The final event batch on tab close or navigation away is sent using the browser's navigator.sendBeacon API rather than a standard fetch call, since sendBeacon is specifically designed to complete delivery even as the page is being torn down — a standard asynchronous request can be cancelled mid-flight in that scenario, silently losing the final events of a session.

Error isolation

All tracking logic is wrapped in try/catch blocks. A JavaScript error inside the tracker — for example, from an unexpected DOM structure on an unfamiliar site — is contained and logged, but never propagates to break the host page.

5.5 Validation Methodology

Rather than relying solely on a single demo site to claim the integration is portable, a second, structurally distinct demo site was built specifically to stress-test this claim: a static-HTML bookstore (deliberately not React, to test framework independence) with a different URL structure (/book/:id rather than /products) and a different funnel sequence.

A verification script was used to simulate a full user session across both sites and confirm, via direct database query, that:

1. Events from each site were correctly tagged with their respective site_id
2. No event from one site appeared under the other site's site_id (no cross-contamination)
3. Funnel stage detection correctly incremented across each site's distinct page sequence, including the dynamic /book/:id path
4. The sendBeacon-based final batch was reliably delivered on tab close for both sites

This validation step is the strongest evidence in the project that the "loosely coupled, plug-and-play" claim is not just an architectural intention but a demonstrated, tested property of the system.

5.6 Client Onboarding Documentation

A one-page integration guide was produced as a client-facing deliverable (included as Appendix A), modelled on the integration documentation patterns used by Google Analytics and Hotjar. It covers the script snippet, the full configuration reference, what is and is not tracked, and a simple verification step a non-technical site owner can follow to confirm correct installation without engineering support.

6. Limitations and Future Work

An honest account of the platform's current limitations is included here deliberately, both because acknowledging boundaries is good engineering practice and because each limitation identified below has a corresponding, scoped piece of future work rather than being an open-ended gap.

6.1 Current Limitations

Limitation	Current impact
No production-grade authentication/authorisation on admin dashboard beyond basic login	Not yet suitable for onboarding real paying clients without hardening
tracker.js hosting is not yet on a production CDN	Demonstrated locally / on a simple static host rather than at production scale and latency
No automated platform plugins (Shopify app, WooCommerce plugin)	Integration currently requires manually pasting a script tag, which is acceptable for technical users but adds friction for non-technical store owners
ML models (XGBoost, NLP sentiment) trained on synthetic/seeded data	Risk and sentiment outputs are illustrative of the pipeline, not yet validated against real customer behaviour at scale
No rate limiting or abuse protection on the ingestion endpoint	A production deployment would need this before accepting traffic from external, untrusted client websites
Single-region, single-database deployment	No high-availability or geographic redundancy considerations have been implemented

6.2 Future Work

Near-term (next development cycle)

- Production-grade authentication for the admin dashboard, including per-client role-based access control
- Deploy tracker.js to an actual CDN with cache headers and versioning, so updates can roll out without requiring clients to change their integration snippet
- Add request validation, rate limiting, and CORS restrictions scoped to registered client domains on the /api/track endpoint
- Retrain ML models against larger, more diverse session datasets — ideally from real pilot clients rather than synthetic seed data

Medium-term

- Build a native Shopify app and WooCommerce plugin that auto-injects the tracking script and a sensible default funnel configuration, removing manual integration entirely for the two largest e-commerce platform ecosystems
- Self-service client onboarding: a sign-up flow that issues a site_id and generates the integration snippet automatically, without engineering involvement
- Session replay (visual playback of a single visitor's journey), a natural extension of the existing heatmap and click-tracking data already being captured
- A/B testing support, allowing a client to compare conversion outcomes across two funnel variants

Long-term

- Real-time alerting — notifying a store owner immediately when a high-value session is flagged as high drop-off risk, rather than only surfacing this retrospectively on the dashboard
- Expansion of the NLP pipeline beyond sentiment scoring to topic extraction (e.g. automatically identifying that negative feedback clusters around "shipping cost" or "checkout complexity")
- Horizontal scaling of the database layer (read replicas, partitioning by site_id) once client volume grows beyond what a single PostgreSQL instance can comfortably serve

7. Conclusion

This project set out to build more than a working analytics pipeline — it aimed to demonstrate that the pipeline could be packaged into something a genuine, independent e-commerce business could adopt with minimal friction. The architecture described in Section 2 proved sound from the outset; the more significant engineering contribution of this project was identifying and resolving the tight coupling that separated a working demo from an onboardable product, documented in Sections 3 and 5.

The resulting system supports multiple independent client websites on a single shared backend, isolates their data correctly, and can be integrated by any site owner through a single script tag — a claim that was tested, not merely asserted, through the construction of a second, structurally distinct demo site. Combined with a business model grounded in a genuine point of competitive differentiation (Section 4), the platform represents a complete answer to the brief: a working, multi-tenant, commercially-positioned customer journey analytics system.

The limitations documented in Section 6 are scoped and specific rather than open-ended, and represent a credible roadmap should this project continue beyond its current academic submission.

Appendix A: Client Integration Guide (As Distributed to Clients)

The following is the client-facing one-page integration guide referenced in Section 5.6, reproduced here in full as it would be distributed to an onboarding client.

How to Integrate

Add behavioural analytics, drop-off prediction, and sentiment insights to your storefront in under five minutes — no SDK, no backend changes, no framework lock-in.

1. Add the Snippet

Paste the following into the <head> of your site, before the closing </head> tag. No other code changes are required.

```
<script>
  window.CJA_CONFIG = {
    site_id: "YOUR_SITE_ID",
    track_endpoint: "https://api.yourplatform.com/api/track",
    funnel: ["/products", "/cart", "/checkout", "/order-complete"]
  };
</script>
<script src="https://cdn.yourplatform.com/tracker.js" async></script>
```

2. Configuration Reference

Parameter	Description
site_id	Your unique client identifier, issued when you sign up. Tags every event so your data stays isolated from other clients.
track_endpoint	The ingestion URL events are sent to. Provided in your onboarding email.
funnel	An ordered array of URL paths describing your conversion journey. Supports partial path matching for dynamic routes such as /product/:id.
debug (optional)	Set to true during setup to log every tracked event to the browser console. Remove before going live.

3. What Gets Tracked

- Page views and route changes (including single-page app navigation)
- Clicks, with on-page coordinates for heatmap generation
- Scroll depth per page
- Funnel progression, mapped automatically against your configured funnel array
- Optional customer feedback text, scored for sentiment

4. Why It's Safe to Install

- Zero dependencies — pure JavaScript, no library conflicts with your existing stack
- Fails silently — a tracking error never breaks your storefront or checkout flow
- Non-blocking — all requests are asynchronous and never delay page rendering
- ~8KB — negligible impact on page load time

5. Verify Your Installation

Set debug: true, open your site, and confirm events tagged [CJA] appear in your browser console as you click and scroll. Then log in to your dashboard — your first sessions should appear within seconds.